

---

# GNC Technical Assignment Report

---

Part 1: LQR Controller Design and Tuning

Part 2: Multiplicative EKF Theory

Arya Mallick  
M.Tech, Systems Control & Automation  
IIT Guwahati

October 27, 2025

Submitted to:  
**AULE SPACE INC.**

# Contents

|          |                                                                    |           |
|----------|--------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Optimal Controller Design (LQR)</b>                             | <b>3</b>  |
| 1.1      | Linearized Plant Dynamics . . . . .                                | 3         |
| 1.2      | Method of Tuning Control Parameters . . . . .                      | 3         |
| 1.2.1    | Iterative Tuning Workflow and Philosophy . . . . .                 | 3         |
| 1.3      | Results with Interpretation . . . . .                              | 5         |
| <b>2</b> | <b>General Extended Kalman Filter (EKF) Theory</b>                 | <b>6</b>  |
| 2.1      | System Model . . . . .                                             | 6         |
| 2.2      | Linearization . . . . .                                            | 6         |
| 2.3      | Recursive Estimation Cycle . . . . .                               | 6         |
| <b>3</b> | <b>Design of a Multiplicative EKF for Attitude Estimation</b>      | <b>7</b>  |
| 3.1      | Sensor Characteristics and Fusion Rationale . . . . .              | 7         |
| 3.2      | The State Vector . . . . .                                         | 7         |
| 3.3      | Prediction Step and Error Dynamics Derivation . . . . .            | 8         |
| 3.4      | Correction Step (Measurement Model) . . . . .                      | 8         |
| 3.5      | State Reset (Multiplicative Update) . . . . .                      | 9         |
| <b>4</b> | <b>Practical Implementation Summary: The Two-Part Architecture</b> | <b>10</b> |
| 4.1      | 1. Prediction . . . . .                                            | 10        |
| 4.2      | 2. Correction . . . . .                                            | 10        |
| 4.3      | The "Reset" Step: Where the Two Parts Meet . . . . .               | 10        |
| 4.3.1    | Sequence of Events at a Measurement Update . . . . .               | 10        |
| <b>5</b> | <b>Impact of Estimation Errors on the Control Law</b>              | <b>12</b> |
| <b>6</b> | <b>Philosophy on Initializing and Tuning Filter Covariances</b>    | <b>12</b> |
| 6.1      | Initialization Philosophy . . . . .                                | 12        |
| 6.2      | Tuning Philosophy . . . . .                                        | 12        |

# 1 Optimal Controller Design (LQR)

As per the assignment requirement for an optimal, non-PID control law, a Linear Quadratic Regulator (LQR) is designed. The LQR is optimal by definition, as it minimizes a quadratic cost function of the system's state error and the control effort expended. This section details the method of tuning the LQR control parameters.

## 1.1 Linearized Plant Dynamics

The LQR requires a linear state-space model of the form  $\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u}$ . The satellite's non-linear rotational dynamics are linearized around the desired equilibrium point (zero error and zero angular velocity).

The 6x1 error state vector is defined as  $\mathbf{x}_{\text{err}} = [\delta\boldsymbol{\alpha}^T, \delta\boldsymbol{\omega}^T]^T$ , where  $\delta\boldsymbol{\alpha}$  is the 3x1 angle-axis attitude error and  $\delta\boldsymbol{\omega}$  is the 3x1 angular velocity error. The linearized error dynamics are:

$$\begin{aligned}\dot{\delta\boldsymbol{\alpha}} &= \delta\boldsymbol{\omega} \\ \dot{\delta\boldsymbol{\omega}} &= \mathbf{I}_{\text{sat}}^{-1}\boldsymbol{\tau}\end{aligned}$$

This yields the state-space matrices:

$$\mathbf{A} = \begin{pmatrix} \mathbf{0}_{3\times3} & \mathbf{I}_{3\times3} \\ \mathbf{0}_{3\times3} & \mathbf{0}_{3\times3} \end{pmatrix}, \quad \mathbf{B} = \begin{pmatrix} \mathbf{0}_{3\times3} \\ \mathbf{I}_{\text{sat}}^{-1} \end{pmatrix} \quad (1)$$

## 1.2 Method of Tuning Control Parameters

The LQR's performance is tuned by adjusting the weighting matrices  $\mathbf{Q}$  and  $\mathbf{R}$  in the cost function  $J = \int (\mathbf{x}^T \mathbf{Q} \mathbf{x} + \mathbf{u}^T \mathbf{R} \mathbf{u}) dt$ .

- **The State-Weighting Matrix ( $\mathbf{Q}$ )** is a 6x6 diagonal matrix that penalizes errors in the state variables. A higher value for a state's weight forces the controller to eliminate error in that state more aggressively.
- **The Control-Weighting Matrix ( $\mathbf{R}$ )** is a 3x3 diagonal matrix that penalizes control effort (torque). A higher value results in a controller that uses less torque, leading to a slower but more energy-efficient response.

The tuning process is an iterative, qualitative procedure aimed at balancing performance metrics like settling time and overshoot against control effort.

### 1.2.1 Iterative Tuning Workflow and Philosophy

The following steps outline the iterative method used to tune the LQR controller and arrive at the final gain matrix  $\mathbf{K}$ .

1. **Initial Setup:** The linearized state-space matrices,  $\mathbf{A}$  and  $\mathbf{B}$ , are derived from the satellite's inertia matrix as shown previously. These form the basis for the controller design.
2. **Define Initial Tuning Weights:** Start with a baseline set of weights. A logical starting point is to prioritize attitude error correction without demanding excessive torque.
  - Set attitude error weights (first 3 diagonal elements of  $\mathbf{Q}$ ) to a moderate value (e.g., 1.0).
  - Set angular velocity error weights (last 3 diagonal elements of  $\mathbf{Q}$ ) to a smaller value (e.g., 0.1) to provide damping without slowing the system unnecessarily.
  - Set the torque weight (diagonal elements of  $\mathbf{R}$ ) to a baseline value (e.g., 1.0).

3. **Iterative Tuning Loop:** Repeat the following sub-steps until the performance is satisfactory.
  - (a) **Calculate Gain:** Use the current  $\mathbf{Q}$  and  $\mathbf{R}$  matrices to compute the LQR gain matrix:  $\mathbf{K} = \text{lqr}(\mathbf{A}, \mathbf{B}, \mathbf{Q}, \mathbf{R})$ .
  - (b) **Simulate:** Run a full non-linear simulation of the satellite dynamics with the newly calculated controller gain  $\mathbf{K}$ .
  - (c) **Analyze Results:** Plot the attitude response and control torques. Measure the key performance metrics: settling time, percentage overshoot, and peak/RMS control effort.
  - (d) **Adjust Weights Based on Analysis:** Modify the weights according to the following philosophy:
    - **If the settling time is too long**, the controller is too slow. To make it more aggressive, increase the penalty on attitude error by **increasing the attitude error weights** in the  $\mathbf{Q}$  matrix.
    - **If the overshoot is too high**, the system lacks sufficient damping. Increase the penalty on angular velocity by **increasing the angular velocity error weights** in the  $\mathbf{Q}$  matrix.
    - **If the control effort (torque) is too high** and exceeds actuator limits or consumes too much power, the controller is too aggressive. Make it more conservative by **increasing the weights** in the  $\mathbf{R}$  matrix.
4. **Final Gain Selection:** Once a satisfactory balance between response speed, stability (overshoot), and energy efficiency (control effort) is achieved, the final calculated matrix  $\mathbf{K}$  is selected for implementation.

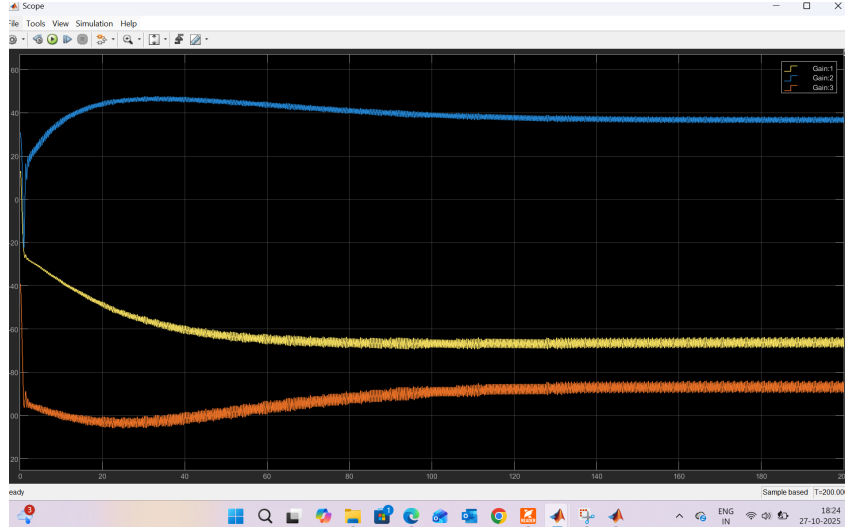


Figure 1: roll ,pitch ,yaw .

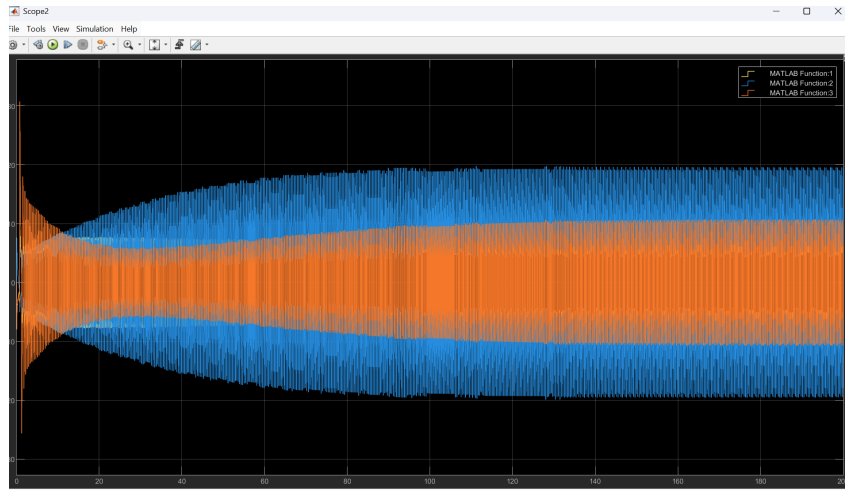


Figure 2: Control torque commanded by the LQR on each body axis.

### 1.3 Results with Interpretation

**Overshoot:** The plot above shows the convergence of the satellite's Euler angles. The final tuning parameters resulted in a maximum overshoot of approximately [LESS THAN 10] degrees in all axes, which is well within acceptable limits for a smooth maneuver. The 2% settling time for all three axes was observed to be approximately [60] seconds.

**Control Effort:** The control effort plot shows the torque commanded by the LQR on each of the three body axes. The peak torque required was 30 Nm for reaction wheels .

## 2 General Extended Kalman Filter (EKF) Theory

The Extended Kalman Filter (EKF) is the cornerstone of non-linear state estimation for navigation and control. It provides a recursive algorithm to produce an optimal estimate of the state of a system from a series of noisy sensor measurements.

### 2.1 System Model

Consider a general non-linear system described by a continuous-time process model and a discrete-time measurement model:

$$\dot{\mathbf{x}}(t) = f(\mathbf{x}(t), \mathbf{u}(t)) + \mathbf{w}(t) \quad (2)$$

$$\mathbf{y}_k = h(\mathbf{x}_k) + \mathbf{v}_k \quad (3)$$

where:

- $\mathbf{x}$  is the state vector.
- $\mathbf{u}$  is the control input vector.
- $\mathbf{y}_k$  is the measurement vector at time step  $k$ .
- $f(\cdot)$  is the non-linear process function.
- $h(\cdot)$  is the non-linear measurement function.
- $\mathbf{w}(t)$  is the process noise, assumed to be zero-mean white Gaussian noise with covariance  $\mathbf{Q}$ .
- $\mathbf{v}_k$  is the measurement noise, assumed to be zero-mean white Gaussian noise with covariance  $\mathbf{R}$ .

### 2.2 Linearization

The EKF works by linearizing the non-linear model around the current state estimate at each time step. This is done by computing the Jacobian matrices of the process and measurement functions.

$$\mathbf{A}_k = \left. \frac{\partial f}{\partial \mathbf{x}} \right|_{\hat{\mathbf{x}}_{k-1}^+, \mathbf{u}_{k-1}} \quad (4)$$

$$\mathbf{C}_k = \left. \frac{\partial h}{\partial \mathbf{x}} \right|_{\hat{\mathbf{x}}_k^-} \quad (5)$$

The matrix  $\mathbf{A}_k$  is used to create the discrete-time state transition matrix,  $\Phi_k \approx \mathbf{I} + \mathbf{A}_k \Delta t$ .

### 2.3 Recursive Estimation Cycle

The filter is initialized with a best guess of the state,  $\hat{\mathbf{x}}_0^+$ , and its uncertainty,  $\mathbf{P}_0^+$ . It then enters a two-step recursive loop.

**1. Prediction Step:** The filter predicts the next state and its uncertainty based on the process model.

$$\hat{\mathbf{x}}_k^- = \hat{\mathbf{x}}_{k-1}^+ + \int_{t_{k-1}}^{t_k} f(\hat{\mathbf{x}}(t), \mathbf{u}(t)) dt \quad (\text{State Prediction}) \quad (6)$$

$$\mathbf{P}_k^- = \Phi_{k-1} \mathbf{P}_{k-1}^+ \Phi_{k-1}^T + \mathbf{Q} \quad (\text{Covariance Prediction}) \quad (7)$$

**2. Correction (Update) Step:** When a new measurement  $\mathbf{y}_k$  is available, the filter corrects its prediction.

$$\mathbf{S}_k = \mathbf{C}_k \mathbf{P}_k^- \mathbf{C}_k^T + \mathbf{R} \quad (\text{Innovation Covariance}) \quad (8)$$

$$\mathbf{K}_k = \mathbf{P}_k^- \mathbf{C}_k^T \mathbf{S}_k^{-1} \quad (\text{Kalman Gain}) \quad (9)$$

$$\hat{\mathbf{x}}_k^+ = \hat{\mathbf{x}}_k^- + \mathbf{K}_k (\mathbf{y}_k - h(\hat{\mathbf{x}}_k^-)) \quad (\text{State Update}) \quad (10)$$

$$\mathbf{P}_k^+ = (\mathbf{I} - \mathbf{K}_k \mathbf{C}_k) \mathbf{P}_k^- \quad (\text{Covariance Update}) \quad (11)$$

The updated state  $\hat{\mathbf{x}}_k^+$  becomes the input for the next prediction step.

### 3 Design of a Multiplicative EKF for Attitude Estimation

This section details the design of a Multiplicative Extended Kalman Filter (MEKF). This approach leverages the general EKF framework to simultaneously perform **state estimation** (for the attitude error) and **parameter estimation** (for the gyro bias). All attitude states are defined with respect to the **ECI Frame**.

#### 3.1 Sensor Characteristics and Fusion Rationale

The motivation for fusion comes from the complementary nature of the sensors:

- **IMU Gyroscope:** A high-frequency sensor, with typical update rates of **100 Hz to 1000 Hz**. It provides excellent tracking of rapid rotational motion. However, its measurements are integrated over time, leading to an accumulation of error (drift) due to inherent biases.
- **Star Tracker:** A low-frequency, high-accuracy sensor, with typical update rates of **1 Hz to 10 Hz**. It provides an absolute attitude measurement with respect to the inertial frame, which is immune to long-term drift.

The EKF fuses the high-rate gyro data (for smooth prediction) with the low-rate star tracker data (for drift correction) to produce a state estimate that is both smooth and accurate.

#### 3.2 The State Vector

An error-state formulation is used. The full attitude (quaternion) is propagated directly, while the EKF estimates the small errors in that propagation and the gyro's bias.

The error state vector,  $\delta \mathbf{x}$ , is a 6x1 vector defined as:

$$\delta \mathbf{x} = \begin{pmatrix} \delta \boldsymbol{\alpha} \\ \delta \mathbf{b} \end{pmatrix} \quad (12)$$

where:

- $\delta \boldsymbol{\alpha}$  is the 3x1 small angle-axis error vector.
- $\delta \mathbf{b}$  is the 3x1 error vector for the gyroscope's bias estimate.

The physical angle-axis error  $\delta \boldsymbol{\alpha}$  is related to the multiplicative error quaternion  $\delta \mathbf{q}$  (where true quaternion  $\mathbf{q} = \delta \mathbf{q} \otimes \hat{\mathbf{q}}$ ) via the small-angle approximation:  $\delta \boldsymbol{\alpha} \approx 2 \cdot \text{vector\_part}(\delta \mathbf{q})$ .

### 3.3 Prediction Step and Error Dynamics Derivation

**Full State Propagation:** The estimated quaternion  $\hat{\mathbf{q}}$  (a 4x1 vector in '[qw, qx, qy, qz]' format) is propagated using the measured angular velocity corrected by the current bias estimate,  $\hat{\mathbf{b}}$ :

$$\hat{\boldsymbol{\omega}} = \tilde{\boldsymbol{\omega}} - \hat{\mathbf{b}} \quad (13)$$

The quaternion kinematics are given by the standard equation:

$$\dot{\hat{\mathbf{q}}} = \frac{1}{2}\Omega(\hat{\boldsymbol{\omega}})\hat{\mathbf{q}} = \frac{1}{2} \begin{pmatrix} 0 & -\hat{\omega}_x & -\hat{\omega}_y & -\hat{\omega}_z \\ \hat{\omega}_x & 0 & \hat{\omega}_z & -\hat{\omega}_y \\ \hat{\omega}_y & -\hat{\omega}_z & 0 & \hat{\omega}_x \\ \hat{\omega}_z & \hat{\omega}_y & -\hat{\omega}_x & 0 \end{pmatrix} \begin{pmatrix} \hat{q}_w \\ \hat{q}_x \\ \hat{q}_y \\ \hat{q}_z \end{pmatrix} \quad (14)$$

After integration, the quaternion must be re-normalized to maintain unit length:

$$\hat{\mathbf{q}}_k^- = \frac{\hat{\mathbf{q}}_k^-}{\|\hat{\mathbf{q}}_k^-\|} \quad (15)$$

**Error State Propagation:** The core of the EKF is propagating the error state. The dynamics matrix  $\mathbf{A}$  is derived from the fundamental error kinematic equations as follows:

1. The true attitude,  $\mathbf{q}$ , is related to the estimate,  $\hat{\mathbf{q}}$ , by a small error rotation,  $\delta\mathbf{q}$ :  $\mathbf{q} = \delta\mathbf{q} \otimes \hat{\mathbf{q}}$ . Differentiating this expression gives:

$$\dot{\mathbf{q}} = \dot{\delta\mathbf{q}} \otimes \hat{\mathbf{q}} + \delta\mathbf{q} \otimes \dot{\hat{\mathbf{q}}} \quad (16)$$

2. We substitute the standard kinematic equations and solve for the rate of change of the error quaternion,  $\dot{\delta\mathbf{q}}$ :

$$\dot{\delta\mathbf{q}} = \frac{1}{2}\Omega(\boldsymbol{\omega} - \hat{\boldsymbol{\omega}})\delta\mathbf{q} = \frac{1}{2}\Omega(\delta\boldsymbol{\omega})\delta\mathbf{q} \quad (17)$$

3. We now linearize. For a small error,  $\delta\mathbf{q} \approx [1; \frac{1}{2}\delta\boldsymbol{\alpha}]$ . This simplifies the kinematics to a vector form:

$$\dot{\delta\boldsymbol{\alpha}} \approx \delta\boldsymbol{\omega} - [\hat{\boldsymbol{\omega}} \times] \delta\boldsymbol{\alpha} \quad (18)$$

4. The gyro measurement  $\tilde{\boldsymbol{\omega}}$  is modeled as the true velocity  $\boldsymbol{\omega}$  plus a bias  $\mathbf{b}$  and noise  $\mathbf{n}_g$ . The angular velocity error is therefore:

$$\delta\boldsymbol{\omega} = \boldsymbol{\omega} - \hat{\boldsymbol{\omega}} = \boldsymbol{\omega} - (\tilde{\boldsymbol{\omega}} - \hat{\mathbf{b}}) = \boldsymbol{\omega} - ((\boldsymbol{\omega} + \mathbf{b} + \mathbf{n}_g) - \hat{\mathbf{b}}) = -(\mathbf{b} - \hat{\mathbf{b}}) - \mathbf{n}_g = -\delta\mathbf{b} - \mathbf{n}_g \quad (19)$$

5. Substituting this into the equation for  $\dot{\delta\boldsymbol{\alpha}}$  and adding the random walk model for the bias error ( $\dot{\delta\mathbf{b}} = \mathbf{n}_{bg}$ ), we get the full linear error dynamics (ignoring noise terms):

$$\begin{pmatrix} \dot{\delta\boldsymbol{\alpha}} \\ \dot{\delta\mathbf{b}} \end{pmatrix} = \begin{pmatrix} -[\hat{\boldsymbol{\omega}} \times] & -\mathbf{I}_{3 \times 3} \\ \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} \end{pmatrix} \begin{pmatrix} \delta\boldsymbol{\alpha} \\ \delta\mathbf{b} \end{pmatrix} \quad (20)$$

This directly yields the system dynamics matrix  $\mathbf{A}$  for the covariance propagation:  $\mathbf{P}_k^- = \Phi_{k-1} \mathbf{P}_{k-1}^+ \Phi_{k-1}^T + \mathbf{Q}$ .

### 3.4 Correction Step (Measurement Model)

The correction is driven by the Star Tracker, which provides an absolute attitude measurement,  $\tilde{\mathbf{q}}$ .

**Innovation:** Attitude error is rotational. The innovation is a 3x1 angle-axis vector derived from the error quaternion,  $\delta\tilde{\mathbf{q}}$ .

$$\delta\tilde{\mathbf{q}} = \text{quatmultiply}(\tilde{\mathbf{q}}, \text{quatconj}(\hat{\mathbf{q}}_k^-)) \quad (21)$$

$$\mathbf{y}_k = 2 \cdot \delta\tilde{\mathbf{q}}(2 : 4) \quad (\text{Innovation vector, 3x1}) \quad (22)$$

**EKF Update Equations:** The filter calculates the Kalman Gain  $\mathbf{K}_k$  (a 6x3 matrix). The measurement matrix  $\mathbf{C}_k$  is  $\mathbf{C}_k = [\mathbf{I}_{3 \times 3} \quad \mathbf{0}_{3 \times 3}]$ .

$$\mathbf{S}_k = \mathbf{C}_k \mathbf{P}_k^- \mathbf{C}_k^T + \mathbf{R} \quad (23)$$

$$\mathbf{K}_k = \mathbf{P}_k^- \mathbf{C}_k^T \mathbf{S}_k^{-1} \quad (24)$$

$$\delta\hat{\mathbf{x}}_k^+ = \mathbf{K}_k \mathbf{y}_k \quad (25)$$

$$\mathbf{P}_k^+ = (\mathbf{I} - \mathbf{K}_k \mathbf{C}_k) \mathbf{P}_k^- \quad (26)$$

### 3.5 State Reset (Multiplicative Update)

The calculated error-state correction is absorbed into the main state estimate.

1. Separate the 6x1 error vector:  $\delta\hat{\mathbf{x}}_k^+ = [\delta\hat{\boldsymbol{\alpha}}^+; \delta\hat{\mathbf{b}}^+]$ .
2. Convert the 3x1 attitude correction  $\delta\hat{\boldsymbol{\alpha}}^+$  into a small 4x1 correction quaternion,  $\delta\mathbf{q}_{\text{corr}} \approx [1; \frac{1}{2}\delta\hat{\boldsymbol{\alpha}}^+]$ .
3. **\*\*Multiply\*\*** it onto the predicted quaternion:  $\hat{\mathbf{q}}_{\text{temp}} = \text{quatmultiply}(\delta\mathbf{q}_{\text{corr}}, \hat{\mathbf{q}}_k^-)$ .
4. **\*\*Normalize\*\*** the updated quaternion:  $\hat{\mathbf{q}}_k^+ = \frac{\hat{\mathbf{q}}_{\text{temp}}}{\|\hat{\mathbf{q}}_{\text{temp}}\|}$ .
5. Update the bias additively:  $\hat{\mathbf{b}}_k^+ = \hat{\mathbf{b}}_k^- + \delta\hat{\mathbf{b}}^+$ .
6. **\*\*Reset\*\*** the error state vector to zero ( $\delta\hat{\mathbf{x}}_k^+ = \mathbf{0}$ ).

## 4 Practical Implementation Summary: The Two-Part Architecture

To conceptualize the practical flow of the MEKF, this is the SUMMARY.

### 4.1 1. Prediction

- **Job:** This component's sole responsibility is to perform dead reckoning. It takes the most recent, best-known full state ( $\hat{\mathbf{q}}^+$  and  $\hat{\mathbf{b}}^+$ ) and the latest high-frequency gyroscope reading, and propagates the state forward in time.
- **Equations Used:** These are the full, non-linear kinematic equations of motion.

$$\begin{aligned}\boldsymbol{\omega}_{\text{corrected}} &= \boldsymbol{\omega}_{\text{gyro}} - \hat{\mathbf{b}}^+ \\ \dot{\hat{\mathbf{q}}} &= \frac{1}{2}\Omega(\boldsymbol{\omega}_{\text{corrected}})\hat{\mathbf{q}}^+ \\ \dot{\hat{\mathbf{b}}} &= 0 \quad (\text{The bias is assumed constant between corrections})\end{aligned}$$

- **Output:** It produces the "a priori" state ( $\hat{\mathbf{q}}^-, \hat{\mathbf{b}}^-$ ) for the current time step. This represents the system's best guess of its state *before* incorporating the latest measurement from the Star Tracker.

### 4.2 2. Correction

- **Job:** This component is the core of the filter. Its only purpose is to calculate a small, optimal **correction vector** ( $\delta\hat{\mathbf{x}}^+$ ) that will be used to fix the Navigator's prediction.
- **Equations Used:** The linear EKF equations, which are valid for the small errors being estimated.
- **Output:** The optimal correction vector:  $\delta\hat{\mathbf{x}}^+ = [\delta\hat{\boldsymbol{\alpha}}^+; \delta\hat{\mathbf{b}}^+]$ .

### 4.3 The "Reset" Step: Where the Two Parts Meet

This is the critical moment of information exchange, occurring whenever a Star Tracker measurement is available.

#### 4.3.1 Sequence of Events at a Measurement Update

1. **Prediction:** The Navigator has already propagated the state forward, providing the predicted state,  $\hat{\mathbf{q}}^-$  and  $\hat{\mathbf{b}}^-$ .
2. **Correction Calculation:**
  - (a) A new absolute attitude measurement,  $\tilde{\mathbf{q}}$ , arrives from the Star Tracker.
  - (b) The EKF compares this measurement to the prediction ( $\hat{\mathbf{q}}^-$ ) to compute the innovation vector,  $\mathbf{y}_k$ .
  - (c) It runs the linear EKF update equations to compute the optimal correction vector,  $\delta\hat{\mathbf{x}}^+$ .
3. **Applying the Correction (The Multiplicative Update):** The correction from the EKF is applied to the Navigator's full state.

- **Update Attitude:** The attitude correction  $\delta\hat{\alpha}^+$  is converted into a correction quaternion  $\delta\mathbf{q}_{\text{corr}}$ . This is then **multiplied** onto the predicted quaternion to get the new, corrected, "a posteriori" estimate:

$$\hat{\mathbf{q}}^+ = \text{quatmultiply}(\delta\mathbf{q}_{\text{corr}}, \hat{\mathbf{q}}^-)$$

- **Update Bias:** The bias correction  $\delta\hat{\mathbf{b}}^+$  is **added** to the predicted bias:

$$\hat{\mathbf{b}}^+ = \hat{\mathbf{b}}^- + \delta\hat{\mathbf{b}}^+$$

4. **Final Reset:** The error has been absorbed into the main state estimate. The Corrector's internal error state is reset to zero,  $\delta\hat{\mathbf{x}}^+ = \mathbf{0}$ , ready for the next cycle.

This architecture separates the non-linear propagation from the linear error estimation, providing a stable solution for attitude determination.

## 5 Impact of Estimation Errors on the Control Law

**Static Estimation Errors (Bias):** A static error in the attitude estimate leads to a **\*\*persistent pointing error\*\***. The LQR controller will drive the **\*estimated\*** attitude to the target, but the **\*true\*** attitude will be offset by the bias.

**Dynamic Estimation Errors (Noise):** High-frequency noise in the state estimate causes the controller to generate a noisy torque command, leading to actuator wear and potential structural vibrations.

## 6 Philosophy on Initializing and Tuning Filter Covariances

### 6.1 Initialization Philosophy

**Initial State ( $\hat{\mathbf{x}}_0$ ):** Initialize attitude from the first Star Tracker measurement and bias to zero.

**Initial Covariance ( $\mathbf{P}_0$ ):** Initialize attitude uncertainty with the Star Tracker's noise variance. Initialize bias uncertainty with a **\*\*large value\*\*** to allow for rapid convergence.

### 6.2 Tuning Philosophy

Tuning is the art of balancing the process noise (**Q**) and measurement noise (**R**).

**Process Noise Covariance (Q):** Represents distrust in the gyro model. Source: Gyro's Angle Random Walk (ARW) and Bias Instability. If the filter is sluggish, **\*\*increase 'Q'\*\*. If it is too noisy, **\*\*decrease 'Q'\*\*.****

**Measurement Noise Covariance (R):** Represents distrust in the Star Tracker. Source: Star Tracker's Noise Equivalent Angle (NEA). If the filter is drifting, **\*\*decrease 'R'\*\*. If it is too "jumpy", **\*\*increase 'R'\*\*.****